시스템 프로그래밍

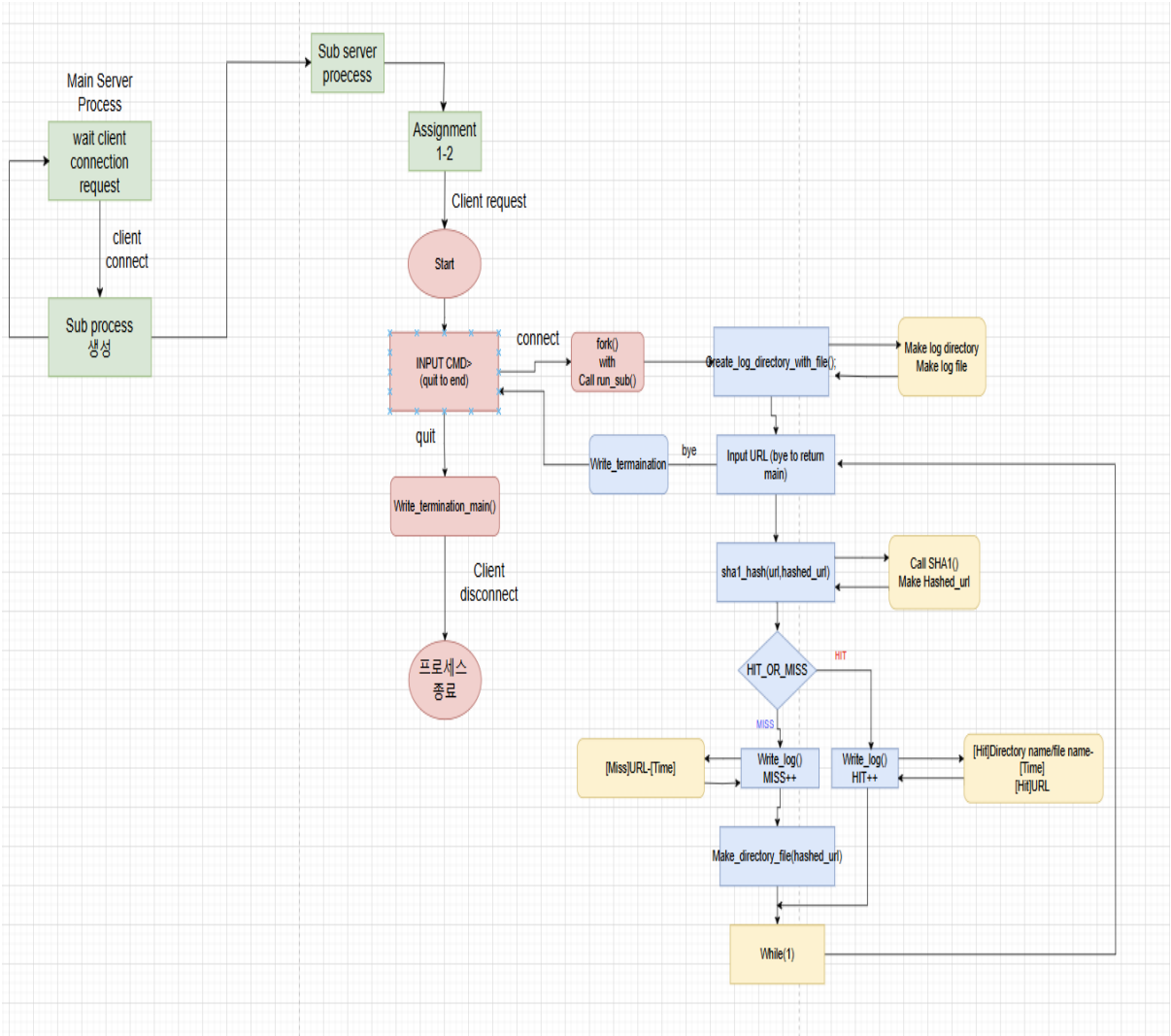
Proxy 2-1

강의 시간	목4 (B)
담당 교수님	최상호 교수님
학번	2021202089
이름	오나균

1) Introduction

이번 과제는 지금까지와는 달리 우선 소스코드를 server.c와 client.c로 나누어야 합니다. 각각은 server와 client를 담당하고, 우선 "./Server"를 실행했을 때, 서버가 열리면, 이 서버에서는 "./Client"를 통해서 다수의 클라이언트에 대해서 URL 요청을 받아서, Main server process의 sub process에서 Assignment 1-2의 연산을 수행하고, Log file에는 getpid()를 통해서 ServerPID에 대한 , Terminated 문구를 작성해줍니다.

2) Flow chart



3) Pseudo code

- server.c

파일 생성 권한 제한을 없애기 위해 umask(0) 설정

서버와 클라이언트 주소 구조체 선언

서버 소켓 생성 (IPv4, TCP)

- 실패하면 에러 메시지 출력 후 종료

서버 주소 구조체 초기화 및 설정 (127.0.0.1, 지정된 포트)

소켓과 주소를 바인딩

- 실패 시 에러 메시지 출력 후 종료

클라이언트 접속을 대기 상태로 설정 (대기 큐 크기 5)

자식 프로세스가 종료되었을 때 좀비 프로세스가 되지 않도록 SIGCHLD 무시

while :

- 클라이언트 접속 수락

  실패하면 에러 메시지 출력 후 종료

- 자식 프로세스를 생성

  자식 프로세스일 경우

  - 서버 소켓 닫기
  - 클라이언트 처리 함수 호출

  부모 프로세스일 경우

  - 자식에게 연결 넘기고 클라이언트 소켓 닫기

루프를 빠져나오면 서버 소켓 닫고 프로그램 종료

- client.c

소켓 생성

- socket() 호출하여 클라이언트 소켓 생성
- 실패 시 에러 출력 후 종료

서버 주소 설정

- server_addr 구조체 설정 (IPv4, 서버 IP: "127.0.0.1", 포트: 40000)

서버와 연결 요청

- connect()로 서버에 연결 시도
- 실패 시 에러 출력 후 종료

사용자로부터 URL 입력 받기 (무한 루프)

- URL을 fgets()로 입력 받음
- 입력받은 URL의 개행 문자 제거 (strcspn())
- 입력받은 URL을 서버에 전송 (write())

bye입력 시 종료

- "bye" 문자열이 입력되면 루프 종료

서버의 응답 받기

- read()로 서버의 응답 읽기
- 응답 출력

소켓 닫기

- 클라이언트 소켓 닫기

4) 결과 화면

- make (server.c ,client.c)

```
kw2021202089@ubuntu:~$ make
gcc -o Server server.c -lcrypto
gcc -o Client client.c -lcrypto
kw2021202089@ubuntu:~$ ./Client
connect: Connection refused
```

⇨ Makefile을 통해서, server.c 와 client.c가 에러없이 컴파일 됩니다.
⇨ 먼저 서버를 작동시켜야 ./Client가 작동한다는 것을 알 수 있습니다.

- 프로젝트 2-1 에서 설명한 예제 수행

```
kw2021202089@ubuntu:~$ ./server
[127.0.0.1 : 43078] client was connected
[127.0.0.1 : 43084] client was connected
[127.0.0.1 : 43084] client was disconnected
[127.0.0.1 : 43078] client was disconnected
^C
```

```
kw2021202089@ubuntu:~$ ./client
input URL> www.kw.ac.kr
MISS
input URL> www.google.com
MISS
input URL> www.naver.com
HIT
input URL> bye
```

```
kw2021202089@ubuntu:~$ ./client
input URL> www.naver.com
MISS
input URL> www.kw.ac.kr
HIT
input URL> bye
kw2021202089@ubuntu:~$
```

⇨ 서버 1개를 작동시키고 두 Client에 대해서 요청을 입력받아서 서버가 연결됩니다.
⇨ 각각의 Client에 대해서, URL 요청을 받아서, Assignment 1-2의 기능을 수행합니다.
⇨ Client 쪽에서 bye 입력시 연결이 종료됩니다.

```
kw2021202089@ubuntu:~$ cat ./logfile/logfile.txt
[Miss] ServerPID : 2227 | www.kw.ac.kr- [2025/5/1, 17:57:07]
[Miss] ServerPID : 2231 | www.naver.com- [2025/5/1, 17:57:11]
[Hit] ServerPID : 2231 | e00/0f293fe62e97369e4b716bb3e78fababf8f90- [2025/5/1, 17:57:14]
[Hit]www.kw.ac.kr
[Terminated] ServerPID : 2231 |  run time: 12 sec. #request hit : 1, miss : 1
[Miss] ServerPID : 2227 | www.google.com- [2025/5/1, 17:57:18]
[Hit] ServerPID : 2227 | fed/818da7395e30442b1dcf45c9b6669d1c0ff6b- [2025/5/1, 17:57:20]
[Hit]www.naver.com
[Terminated] ServerPID : 2227 |  run time: 21 sec. #request hit : 1, miss : 2
```

⇨ /logfile/logfile.txt의 경로에 적힌 결과를 확인했을 때, Assignment1-2의 기능이 잘 수행되었으며, 과제에서 설명한 예제와 마찬가지로 구현된 것을 확인 할 수 있습니다.

5) 고찰

이번 과제는 이제 server와 client에 대해서 각각 코드를 작성하고, 이 둘 사이에서 socket을 사용하여, 간단하게 서버와 클라이언트 사이에서의 통신을 구현하는 것이었는데, 처음에는 제가 실습자료가 있는 것을 까먹고, 그냥 실습자료를 안 보고 하였는데, 그래서 초반에는 이론 시간의 시험 범위 자료에 server 와 client에 대한 코드가 있어서 이것을 보면서 하였습니다.

그러다가 client.c 에서 서버에 URL을 보내는 부분인 send(cd, buf, sizeof(buf), 0); 로 처음에 이렇게 작성하였다가, MISS 또는 HIT이 두번씩 출력되었는데, 이에 대해서 찾아보니까 sizeof를 쓰면 버퍼 전체 크기 (제 코드에서는 1024) 를 전송하여서, 이로 인해 두 번 출력되는 현상이 일어나는 것이었습니다. 그래서 이를 send(cd, buf, strlen(buf), 0)로 수정하여서, 이제 정상적으로 1번씩만 출력되게 바꾸었습니다.

근데 이제 제가 뒤늦게 실습자료가 있는 것을 깨달아가지고, 실습자료를 보니까 간단하게 write와 read를 사용해서, 이번 과제에 대한 코드가 작성되어있는 것을 보고, 허무감을 느꼈지만, 그래도 잘 참고해서 마저 코드를 작성할 수 있었습니다.

뭔가 이번 시스템프로그래밍 수업의 과제들은 2학년 때의 전공과 달리 실습자료에서 유용한 정보를 굉장히 많이 얻을 수 있는 것 같습니다.

6) Reference